

MagicCode Super-Resolution Software Getting Started

Document Version: v2.2.0. Release Date: September 30, 2026.

1. Choose an integration path

Path	Library	Header	Use when
Spatial (CPU or GPU)	libmagic_sr.a / libmagic_sr.lib	mc_interface.h	mc_nscaler_enable per frame; NULL-handle create defaults to 2x SPATIAL_SPEED
Temporal (GPU)	libmagic_sr.a / libmagic_sr.lib	mc_interface.h	ctrl_param_t create + temporal_frame_t per frame
Linux x86_64	lib/linux/libmagic_sr.a	mc_interface.h	CPU spatial archive (AVX2); no GPU backend objects

2. Spatial quick start (nscaler)

Link libmagic_sr, include mc_interface.h, bind per-frame input/output, then (a NULL handle creates a default 2x SPATIAL_SPEED session):

```
void* h = NULL; mc_nscaler_enable(&h, &in_frame, &out_frame); /* then mc_nscaler_disable(h) */
```

- Scale [1.0, 8.0]; <=0 defaults to 2.0.
- Android GLES/Vulkan: bind width/height on mc_nscaler_input_frame_t; the first enable sets the handle size (no size-hint call in v2.2.0).
- v2.2.0 removed the Enable archive and mc_enable.h. Non-default backend/scale/model: create with mc_nscaler_control(MC_NSCLER_CMD_SET_PARAM).

3. Temporal session (v2.2.0)

Link libmagic_sr, include mc_interface.h, create with

mc_nscaler_control(MC_NSCLER_CMD_SET_PARAM) on a ctrl_param_t (alg_mode = TEMPORAL_SPEED_MODE or TEMPORAL_BALANCED_MODE, gpu_context, create-stage depth/HDR), then each frame fill mc_nscaler_input_frame_t / mc_nscaler_output_frame_t plus a temporal_frame_t (struct_size, depth, motion, jitter, frame_index, camera) and call mc_nscaler_enable.

Temporal Super-Resolution

Temporal super-resolution runs through the nscaler session API: libmagic_sr.a (Windows: libmagic_sr.lib) + mc_interface.h + mc_nscaler_enable / mc_nscaler_control / mc_nscaler_disable. v2.2.0 replaced the v2.0.0 MC_Init / MC_Process / MC_Uninit entry and removed the v2.1.0 MC_Enable helper (mc_enable.h is deleted). A NULL *handle on mc_nscaler_enable creates a session with defaults (2x SPATIAL_SPEED) and processes the first frame.

Modes

Value	Enumerator	Role
0	SPATIAL_SPEED_MODE	Spatial, throughput first
1	SPATIAL_BALANCED_MODE	Spatial, quality/cost trade-off
2	TEMPORAL_SPEED_MODE	Temporal, throughput first
3	TEMPORAL_BALANCED_MODE	Temporal, canonical FSR-family path

- Temporal scaler_factor is (1, 8]; 1.0 itself is rejected. Spatial remains [1.0, 8.0].
- TEMPORAL_SPEED_MODE on Metal, Vulkan, and OpenGLS currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family temporal path (same family as Balanced, not a separate customer API).
- TEMPORAL_BALANCED_MODE uses the canonical FSR-family path on every GPU temporal backend. Desktop Balanced edge-resolve is supported on Windows Vulkan, Windows Direct3D 11, desktop OpenGL 4.3, and Apple Metal. Android Vulkan and Android OpenGLS do not run edge-resolve. If enable_sharpening is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run temporal modes.
- The API accepts (1, 8]. Exact 2× has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- mc_nscaler_input_frame_t / mc_nscaler_output_frame_t: caller-owned color and output GPU resources.
- mc_nscaler_input_frame_t.frame: non-NULL temporal_frame_t* with struct_size = sizeof(temporal_frame_t).
- temporal_frame_t.depth and motion at input resolution.
- jitter_offset_x/y in input pixels, top-left, +X right, +Y down.
- frame_index / reset_history; camera_near / camera_far / camera_fov_y (radians).
- gpu_context at create (mc_nscaler_control SET_PARAM, or the first mc_nscaler_enable): Vulkan requires physical_device + device; D3D11 requires device; Metal device may be NULL; GL/GLES use the current context (library never makes current).
- Vulkan temporal requires mc_nscaler_input_frame_t.command_buffer to be a recording VkCommandBuffer. Metal command_buffer is optional. D3D11 / GL / GLES leave it NULL.

- Create-stage depth_reversed / depth_infinite / hdr_color on ctrl_param_t (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.
- motion_vector_scale_x/y is the only MV numeric conversion. (0,0) defaults to (input_width, input_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- reactive / transparency are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both temporal modes enable depth+MV history rejection by default.
- Diagnostic only: MAGICSR_TAAU_AUTO_MASK=0 and MAGICSR_TAAU_HIST_REJECT=0 can turn those defaults off. They are not a customer configuration path.

4. Models

GPU model bins are magic_<backend>_speed_gpu_params.bin and magic_<backend>_balanced_gpu_params.bin. Use tools/setup_models.sh (copies Speed + Balanced GPU bins). Set ctrl_param_t.model_path at create in production apps. Linux CPU spatial uses the CPU model bin (magic_sr_cpu_params.bin).

5. Linux platform library

- lib/linux/libmagic_sr.a ships the x86_64 CPU spatial path (AVX2/FMA, POSIX threads). Rebuild with tools/build_linux_lib.sh; link with -lm -lpthread.
- The shipped Linux archive contains no GPU backend objects. Linux Vulkan temporal (FSR-family Speed/Balanced) builds from source; tools/linux_vulkan_temporal_itest.c is a headless reference integration validated on lavapipe with output matching the Windows NVIDIA reference.
- CPU temporal is not available on any platform, including Linux.

6. Next documents

- User Guide: doc/User_Guide.md and doc/用户使用说明书.md
- API Reference, Integration Guide, FAQ, Migration Guide, Known Issues in this folder